

MULTITHREADING TUTORIAL

ADVANCED PROGRAMMING

Threads are a way of parallelizing our program so that instructions which can execute simultaneously do not have to wait for each other. There are 2 ways of parallelizing execution:

- 1) Extending the Thread Class
- 2) Implementing the Runnable Interface

NOTE: When different threads access a shared resource like a file, race conditions can occur. These happen because multiple users are trying to read/write a particular value. Let us explain this through an example of a movie ticket booking. If 3 tickets are available, and 2 different users access the movie booking website simultaneously (assume both want to book 2 tickets each), then only 1 booking will be successful. Both requests cannot be successful, since 4 seats are to be booked when only 3 are available. Only 1 request can be successful. Hence when using threads, we need to apply locks so that only 1 thread can read/write from a shared resource. Though this problem is studied in depth in OS, for our purpose it is sufficient to just handle this challenge in Java. For that, we use the 'synchronized' keyword. To make a method 'synchronized', use the following syntax:

public synchronized void funcName(){}

This will ensure that only one thread can run at any time; the other thread has to just wait.

Q) In this question, we will make use of multithreading to implement an online bus reservation system. In a remote system, multiple users can access the system at the same time to make their bookings. Each such connection of a user with the system will be handled by a different thread. To accomplish this, proceed in the following manner:

- 1) Create a TicketCounter class, which has 1 attribute availableSeats (you can directly set this variable to value 3). The class also has a method bookTicket, which takes the user name (String) and number of seats that they want to book (Integer) as arguments. If the number of seats to be booked is less than equal to the available seats, then this method makes the booking. Otherwise, it prints that seats are not available. The method bookTicket() is called by another method, displaySeats(). This method takes the same 2 parameters, name and number of seats. It first prints the number of seats currently available, and then proceeds to the booking stage by calling the bookTicket() method.
- 2) Next define a TicketBookingThread class that extends from the Thread class. This class will have 3 attributes: an object of the TicketCounter class, a string

name, and an integer noOfSeats. Make a constructor to initialize these values. In the run() method, you just have to call the displaySeats() method of the TicketCounter class using the initialized values.

- 3) In the main() method, first create an object of the TicketCounter class. Use this object to create 2 objects of the TicketBookingThread class (basically 2 threads are initialized). You can use random names for the String attribute, and take 2 as the value to be passed as the integer attribute (both threads are aiming to book 2 seats). Call the start() methods on both threads and run the program.

SOLUTION:

```
import java.util.*;

class TicketCounter{
    private int availSeats = 3;

    public void bookTicket(String name, int seats){
        if(seats<=availSeats && seats>0){
            System.out.println("Hello "+name+": "+seats+" seats booked successfully.");
            availSeats-= seats;
        }
        else{
            System.out.println("Sorry "+name+": some seats were booked just before you, and we do not have enough vacancies now.");
        }
    }

    public void displaySeats(String name, int seats){
        System.out.println("Hello "+name+", "+availSeats+" seats are available. Proceeding with your booking.");
        this.bookTicket(name, seats);
    }
}

class TicketBookingThread extends Thread{
    private TicketCounter ticketcounter;
    private String pName;
    private int numSeats;
```

```

    public TicketBookingThread(TicketCounter tc, String n, int ns){
        ticketcounter = tc;
        pName = n;
        numSeats = ns;
    }

    public void run(){
        ticketcounter.displaySeats(pName, numSeats);
    }
}

public class Classroom2{
    public static void main(String[] args){
        TicketCounter tickCount = new TicketCounter();
        TicketBookingThread t1 = new TicketBookingThread(tickCount, "Arya",
2);
        TicketBookingThread t2 = new TicketBookingThread(tickCount,
"Mayank", 2);

        t1.start();
        t2.start();
    }
}

```

Q) On getting the output, do you see the issues that were highlighted above? With 3 seats, would 2 users be able to book 2 seats each (if they access the system simultaneously)? Edit the code, so that this issue is resolved, and only 1 of the bookings is successful.

SOLUTION: Add the synchronized keyword in the bookTicket method. This will ensure that both threads cannot read from the system at the same time (by using locks). Hence once one thread has finished, then only the other thread can start running this method.

```

public synchronized void bookTicket(String name, int seats){
    if(seats<=availSeats && seats>0){
        System.out.println("Hello "+name+": "+seats+" seats booked
successfully.");
        availSeats-= seats;
    }
    else{

```

```
        System.out.println("Sorry "+name+": We don't have the requested  
number of seats available any more.");  
    }  
}
```

To read more about synchronized, you refer to the following resources:

- 1) [Synchronization in Java - GeeksforGeeks](#)
- 2) [Guide to the Synchronized Keyword in Java | Baeldung](#)

Extra Exercise: You can try executing the same system by implementing the Runnable interface rather than extending the Thread class.